

# Summary

The primary objective of this session is to master the calculation of the minimal number of states in a DFA required to accept languages defined by specific regular expressions. The core takeaways include:

- **State Count Dynamics:** Minimal state counts are not determined by the length of the expression alone but by the specific string lengths the language accepts or rejects.
- **The “AD Rule”:** A set of heuristics and algebraic shortcuts, including the use of LCM for periodic constraints, to determine state counts without constructing the full DFA.
- **Alphabet Sensitivity:** The size of the input alphabet ( $\Sigma$ ) significantly impacts state counts. If a symbol in  $\Sigma$  is not mentioned in the regular expression, the DFA requires a dead state to remain complete.
- **Structural Equivalency:** Regular expressions are conceptually similar to NFAs because they focus on generating valid strings, whereas DFAs must explicitly handle rejection paths.

## Minimal DFA State Counting: Case Studies

### Periodic and Star-Closure Expressions

When analyzing expressions of the form  $(a^m + a^n)^*$ , the objective is to identify the Frobenius coin boundary—the smallest integer beyond which all string lengths become generable.

| Regular Expression | Accepted String Lengths (n)    | Minimal States | Key Logic                                                                                        |
|--------------------|--------------------------------|----------------|--------------------------------------------------------------------------------------------------|
| $(aa + aaa)^*$     | 0, 2, 3, 4, 5, ...             | 3              | $Q_0$ (final) accepts $\epsilon$ ; $Q_1$ handles $a^1$ (reject); $Q_2$ handles $a^2$ and beyond. |
| $(aa + aaaa)^*$    | 0, 2, 4, 6, ... (even lengths) | 2              | Standard parity tracking for even numbers.                                                       |
| $(1 + 111)^*$      | 0, 1, 2, 3, ... (all lengths)  | 1              | Single-state loop accepting all unary strings.                                                   |
| $(aaa + aaaaa)^*$  | 0, 3, 5, 6, 8, 9, ...          | 9              | Rejects 1, 2, 4, 7; requires tracking up to the periodic boundary.                               |

|                  |                    |   |                                                        |
|------------------|--------------------|---|--------------------------------------------------------|
| $(11 + 11111)^*$ | 0, 2, 4, 5, 6, ... | 5 | Rejects 1 and 3; periodic behavior after initial gaps. |
|------------------|--------------------|---|--------------------------------------------------------|

The general principle is that minimal states depend on the largest unreachable length before periodicity begins.

## Mathematical Constraints on n

Languages defined as  $a^n$  where  $n = ax + by$  ( $x, y \geq 0$ ) require state tracking based on unreachable lengths before periodic repetition.

### Example 1:

$$L = \{ a^{(3x + 4y)} \mid x, y \geq 0 \}$$

Generated lengths: 0, 3, 4, 6, 7, 8, ...

Missing lengths: 1, 2, 5

Result: 7 states are required.

The DFA must distinguish lengths up to the Frobenius number before continuous acceptance begins.

### Example 2:

$$L = \{ a^{(2x + 3y)} \mid x, y \geq 0 \}$$

Generated lengths: 0, 2, 3, 4, 5, ...

Missing length: 1

Result: 3 states are sufficient.

Tracking is required only until the first unreachable length.

## Union and Intersection Periodicities

The AD Rule applies LCM for modular periodic constraints.

- Union Condition (multiple of 2 OR 3):**  
 Periodicity repeats every  $\text{LCM}(2, 3) = 6$ .  
 Minimal DFA requires 6 states.
- Intersection Condition (multiple of 2 AND 3):**  
 Equivalent to multiples of 6.  
 DFA must track modulo 6.  
 Minimal DFA requires 6 states.

## Impact of Alphabet Size on DFA Completeness

A DFA must be complete over its alphabet  $\Sigma$ .

**Scenario:**  $RE = (1 + 111)^*$  over  $\Sigma = \{0, 1\}$

- If  $\Sigma = \{1\}$ , the language includes all unary strings of 1.  
Minimal DFA: 1 state.
- If  $\Sigma = \{0, 1\}$ , the symbol 0 must be handled explicitly.  
Any occurrence of 0 transitions to a dead state.  
Minimal DFA: 2 states (1 accepting loop + 1 dead state).

This highlights that minimal state count depends not only on the language structure but also on alphabet definition.

## Regular Expression Construction Principles

Regular expressions describe sets of valid strings and are inherently non-deterministic.

For  $\Sigma = \{a, b\}$ , common patterns include:

| Language Description             | Regular Expression     |
|----------------------------------|------------------------|
| All strings including $\epsilon$ | $(a + b)^*$            |
| All strings excluding $\epsilon$ | $(a + b)^+$            |
| Strings starting with a          | $a(a + b)^*$           |
| Strings ending with ab           | $(a + b)^*ab$          |
| Strings containing substring ab  | $(a + b)^*ab(a + b)^*$ |

## Substring Logic and DFA Variations

The standard NFA-style expression for substring “ab” is:

$(a + b)^*ab(a + b)^*$

However, a deterministic-style regular expression may impose structural order, such as:

$b^*a^*ab(a + b)^*$

Both expressions represent the same language but reflect different construction strategies.

## Strategic Insights for Competitive Preparation

### Relationship Between $L$ and $L^c$

The minimal DFA for a language  $L$  and its complement  $L^c$  require the same number of states. The transformation is achieved by swapping final and non-final states.

Application:

If the minimal DFA for substring “0111” requires  $n + 1$  states, the complement also requires  $n + 1$  states.

### Heuristics for Pattern-Based Languages

- **Ends-With Rule:**  
For a pattern of length  $n$ , minimal DFA states =  $n + 1$ .
- **Substring Rule:**  
For a substring of length  $n$ , minimal DFA states =  $n + 1$ .  
The  $(n + 1)$ th state is final and typically contains a self-loop.

## Pedagogical Emphasis

The session emphasizes patience as the foundation of mastery. Effective preparation is defined by conceptual depth rather than speed. Sustainable success in Theory of Computation requires structural clarity, not short-term motivational slogans. The instructional philosophy prioritizes long-term technical mastery over accelerated completion claims.